



ÉCOLE CENTRALE CASABLANCA

DATA ENGINEERING

INGESTION PYTHON, STRESS TESTING, RE-ENGINEERING
DBT, INCREMENTAL, SCD2 ET QUALITE DES DONNEES

Labs 1 et 2 – Pipeline Data Engineering Play Store

Élève :

Abderrazzak OUTZOULA
Badreddine Saadioui

Enseignant :

Enseignant du module

Table des matières

Resume	2
1 Plan du rapport et fil conducteur	3
1.1 Objectifs de ce rapport	3
1.2 Methode appliquee pour ce rapport	3
2 Lab 1 - Pipeline Python et stress testing	4
2.1 Objectif et structure du Lab 1	4
2.2 Qualite de base du pipeline avant stress test	4
2.3 Figure Lab 1 : dashboard de synthese	5
2.4 Stress testing (Partie C) : protocole et resultats	5
2.5 Reponses explicites aux questions de la Partie C	7
3 Lab 2 - Re-engineering dbt + DuckDB	7
3.1 Objectif du Lab 2	7
3.2 Design data model (Kimball) et grain	7
3.3 Staging : nettoyage et deduplication robuste	8
3.4 Figure Lab 2 : top applications	9
3.5 Figure Lab 2 : tendance mensuelle	9
4 Implementation detaillee des points techniques critiques	10
4.1 Incremental loading (implementation + preuve)	10
4.2 SCD2 snapshot (implementation + preuve)	11
4.3 Data quality & testing (implementation + preuve)	11
4.4 Volumes modeles et coherence de grain	12
5 Comparaison pipeline Python-only vs pipeline dbt	13
6 Reflexions critiques sur nos choix d'architecture	13
6.1 Partie la plus fragile de la pipeline	13
6.2 Plus grand insight d'architecture	13
6.3 Une decision de design que nous changerions	13
7 Tracabilite : modeles, sorties et preuves utilisees	14
7.1 Modeles et scripts references	14
7.2 Fichiers de preuve utilises dans ce rapport	14
8 Conclusion generale	14
References	15

Resume

Ce rapport presente notre rendu final pour les deux labs. Nous avons volontairement gardé le fil logique du projet : d'abord un pipeline Python complet (Lab 1), puis son re-engineering avec dbt et DuckDB (Lab 2). L'objectif est de documenter clairement ce que nous avons implemente, les choix techniques retenus, les resultats obtenus, et la lecture detaillee des figures a partir de nos propres sorties.

1 Plan du rapport et fil conducteur

1.1 Objectifs de ce rapport

Les objectifs principaux de cette version sont :

- decrir l'architecture finale (et les evolutions ajoutes),
- expliquer l'implementation de l'incremental loading, du SCD2 et des tests qualite,
- proposer une comparaison Python-only vs dbt,
- repondre aux trois questions de reflection (partie la plus fragile, insight d'architecture, decision a changer),
- appuyer toutes les affirmations par des references explicites vers nos modeles et nos sorties,
- inserer de vraies captures/resultats et les commenter.

1.2 Methode appliquee pour ce rapport

Pour etre rigoureux, nous avons suivi trois regles de redaction :

- chaque section contient des chemins de fichiers precis,
- chaque figure est suivie d'une lecture metier (pas seulement une legende),
- chaque exigence est tracee a une sortie reelle (CSV, JSON, image ou modele SQL).

Point couvert	Ce que nous avons fait	Ou verifier
Architecture finale	Separation claire entre ingestion Python, staging dbt, marts, snapshot et qualite.	Figure 4; modeles dbt
Incremental loading	Table de faits incrementale avec filtre temporel sur <code>review_ts</code> .	Figure 7; <code>lab_report_delta_summary.json</code>
SCD2	Snapshot dbt pour historiser les changements des applications.	Figure 8; <code>lab_report_scd2_target_app.csv</code>
Data quality et tests	Tests de schema, tests metier et integrite relationnelle (<code>relationships</code>).	Figure 9; <code>run_results.json</code>
Comparaison Python vs dbt	Comparaison des deux approches sur notre cas reel.	Section 5
Reflexions	Retour critique sur fragilites, apprentissages et choix de design.	Section 6

TABLE 1 – Matrice de couverture des objectifs du rapport.

2 Lab 1 - Pipeline Python et stress testing

2.1 Objectif et structure du Lab 1

Le Lab 1 visait a construire un pipeline de bout en bout pour les donnees Google Play avec quatre etapes : ingestion, transformation, serving, dashboard. Les scripts principaux sont :

- `src/ingest.py`
- `src/transform.py`
- `src/serve.py`
- `src/dashboard.py`
- `src/stress_test.py`

Les sorties produites dans `data/processed/` ont constitue la base de l'analyse metier : `apps.csv`, `reviews.csv`, `quality_report.json`, `app_kpis.csv`, `daily_kpis.csv`, `dashboard.png`, `stress_test_report.md`.

2.2 Qualite de base du pipeline avant stress test

Sur le run de base, `quality_report.json` indique un pipeline propre : 18898 reviews en entree, 18898 en sortie, 0 doublon `review_id`, 0 `unknown_app_id`, 0 score invalide, 0 timestamp invalide (une seule review vide en contenu texte, tracee explicitement).

Ce point est important : nous n'avons pas commence le stress testing sur un pipeline deja instable.

2.3 Figure Lab 1 : dashboard de synthese

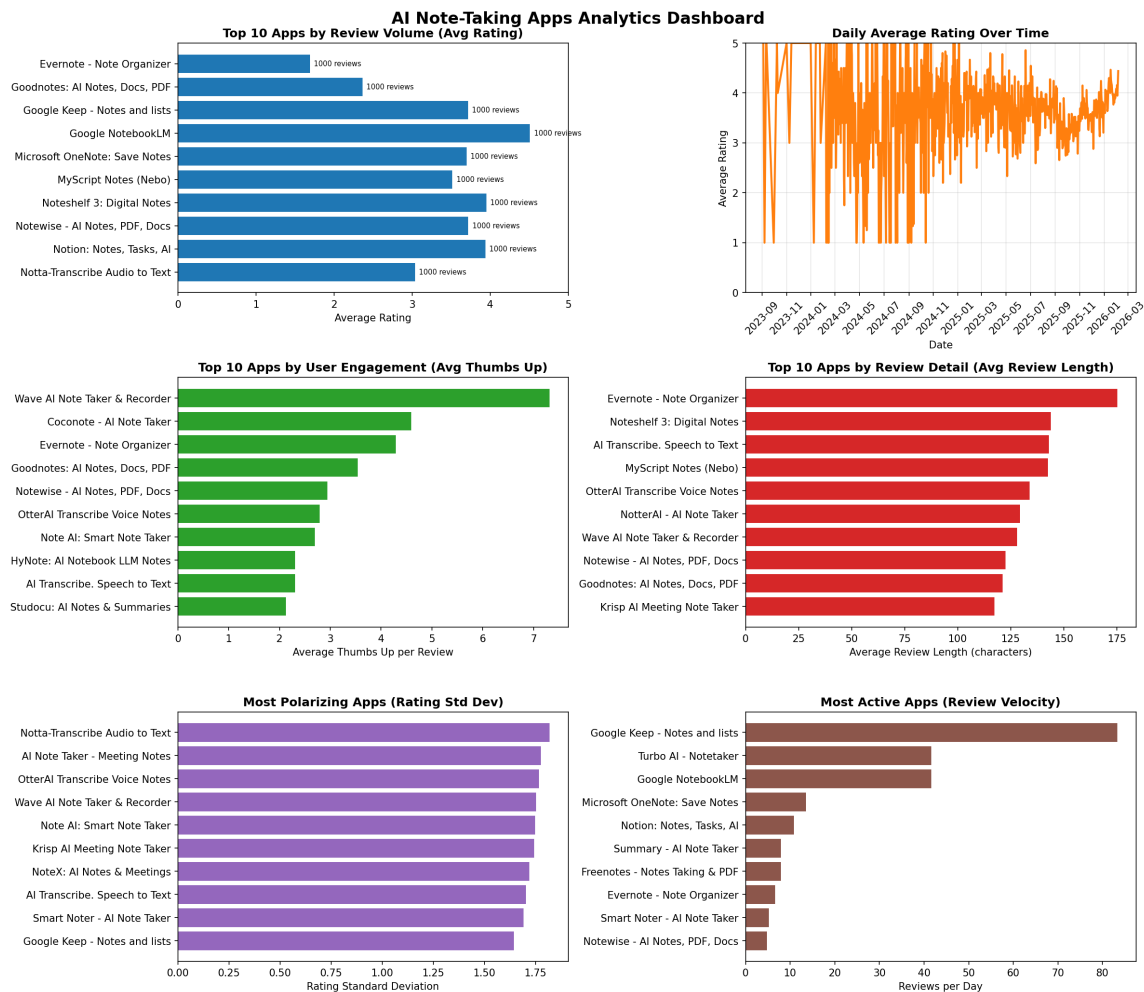


FIGURE 1 – Dashboard Lab 1 (volume, note moyenne, dynamique journaliere et engagement).

Lecture detaillee : cette figure condense les KPI utiles pour piloter la qualite percue des applications. Nous avons volontairement combine des indicateurs de volume (nombre d'avis), de niveau (note moyenne) et de dynamique (evolution temporelle) pour eviter des conclusions trompeuses basees sur un seul axe. Par exemple, une app peut avoir une note elevee mais sur peu d'avis ; a l'inverse, une app massive peut avoir un score moyen tire vers le bas par des vagues de feedback negatif.

2.4 Stress testing (Partie C) : protocole et resultats

Nous avons execute les 5 scenarios demandes dans la consigne de Lab 1 et consigne toutes les observations dans `data/processed/stress_test_report.md`.

Scenario	Statut	Duplicates reviews	Unknown apps	Invalid scores/timestamps
1) New Reviews Batch	ok	1	10	0 / 0
2) Schema Drift in Reviews	ok	0	10	0 / 0
3) Dirty & Inconsistent Records	ok	0	10	3 / 1
4) Updated Applications Metadata	ok	0	18898	0 / 0
5) New Business Logic	ok	0	0	mismatch detecte

TABLE 2 – Resume quantifie des scenarios de stress test.

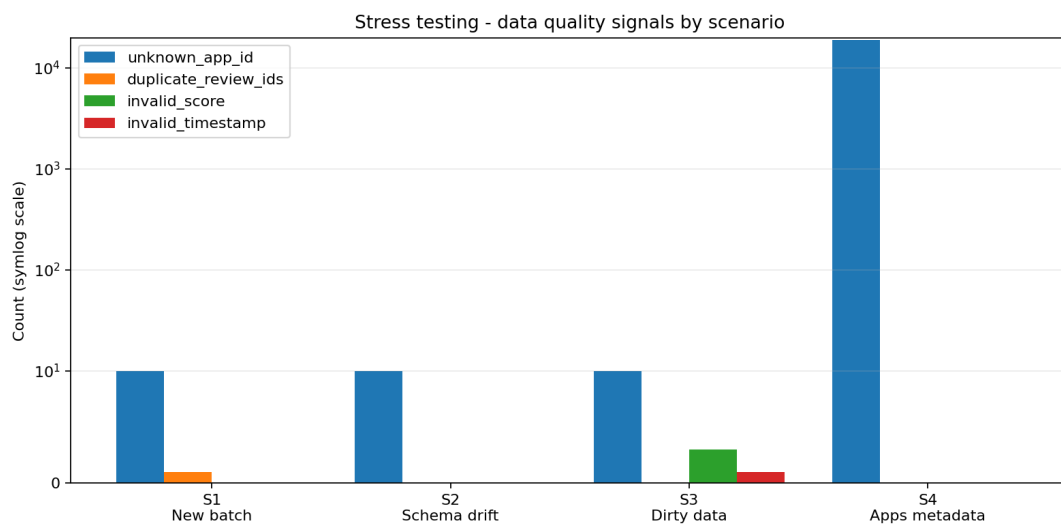


FIGURE 2 – Stress testing - signaux de qualite par scenario.

Lecture detaillee : la figure montre que les scenarios 1 a 3 restent localement degrades mais gerables, alors que le scenario 4 provoque une rupture forte de referentiel (`unknown_app_id` = 18898). C'est exactement le type de défaut que le stress test doit reveler : le pipeline continue de tourner, mais il rend visible que la jointure metier devient non fiable.

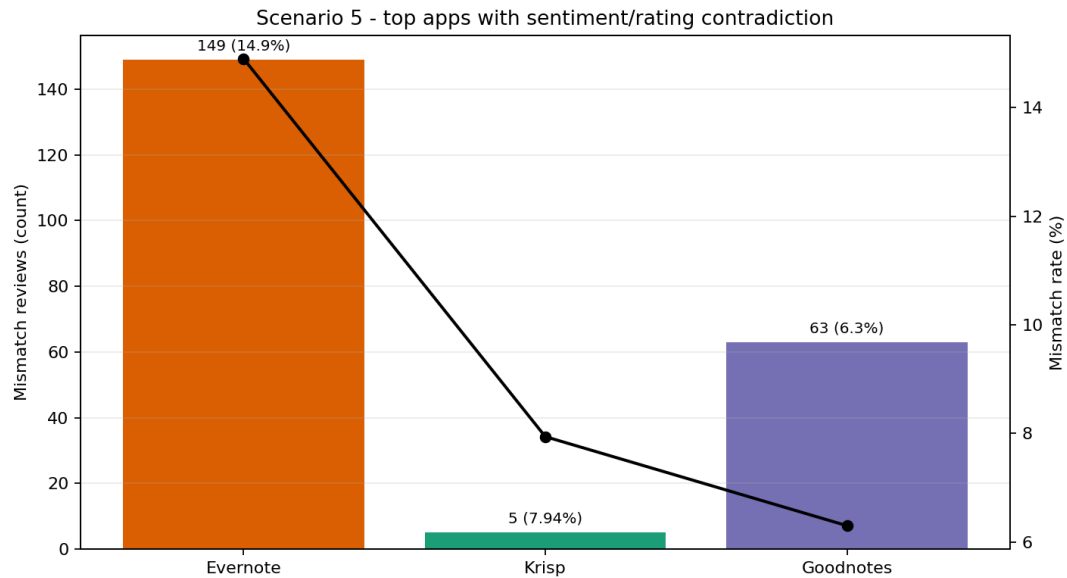


FIGURE 3 – Stress testing - contradictions texte vs note (scenario 5).

Lecture detaillee : cette figure repond a la question metier du scenario 5. Nous avons detecte 831 reviews en contradiction sur 30 apps analysees (27 apps touchees). La lecture correcte est double : volume absolu et taux relatif. Exemple concret : Evernote concentre 149 contradictions (volume fort), alors que Krisp a un taux plus eleve mais sur un volume faible.

2.5 Reponses explicites aux questions de la Partie C

- **Scenario 1 :** les doublons `review_id` sont bien dedoublonnes (10 entrees -> 9 lignes).
- **Scenario 2 :** le schema drift est absorbe via mapping des colonnes et parsing date adapte.
- **Scenario 3 :** les valeurs invalides sont neutralisees sans casser le run, et comptees.
- **Scenario 4 :** l'unicite `appId` reste geree, mais le decrochage de referentiel est remonte explicitement.
- **Scenario 5 :** la nouvelle logique metier est integree dans la serving layer sans casser le pipeline global.

3 Lab 2 - Re-engineering dbt + DuckDB

3.1 Objectif du Lab 2

Dans le Lab 2, nous avons conserve l'ingestion Python du Lab 1 et migre toute la transformation vers dbt/DuckDB pour standardiser la pipeline selon une architecture analytique moderne : couche staging, couche marts, tests natifs et historisation SCD2.

3.2 Design data model (Kimball) et grain

Nous avons formalisé le processus metier autour de l'evenement *review*. Le grain de la table de faits est :

Une ligne de la fact represente une review unique.

Dimensions implementees :

- `dim_apps` (contexte applicatif),
- `dim_dates` (axe temporel),
- `dim_reviewers` (auteurs dedoublonnes).

Table de faits :

- `fct_playstore_reviews` avec mesures `rating_score`, `thumbs_up_count`, `review_count`, `low_rating_flag`.

Architecture Finale Labs 1 et 2

Pipeline réel: ingestion Python puis industrialisation avec dbt (incremental, SCD2 et tests).

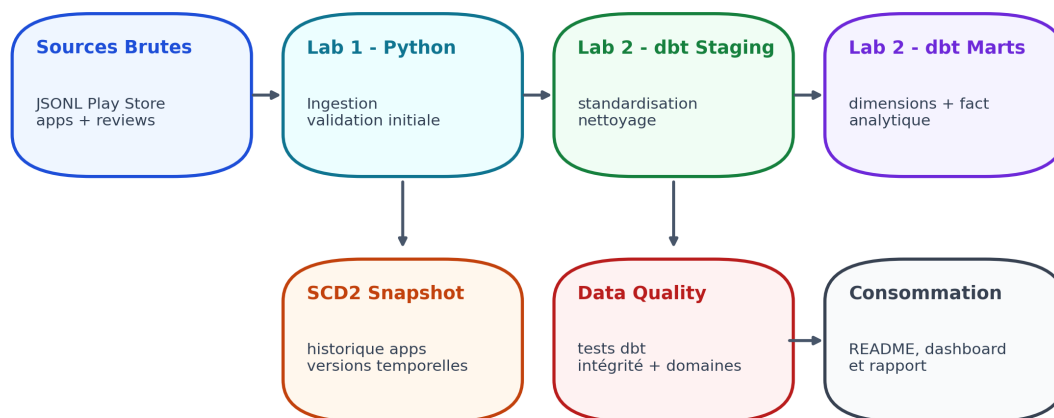


FIGURE 4 – Architecture finale Labs 1+2 (ingestion Python, dbt staging/marts, SCD2, testing).

Lecture detaillee : cette figure illustre la separation qui manquait en Lab 1 entre extraction et transformation. L'ingestion reste simple et robuste en Python, pendant que dbt prend en charge la normalisation SQL, la modélisation analytique et les controles qualite.

3.3 Staging : nettoyage et deduplication robuste

Le modele `stg_playstore_apps.sql` inclut une deduplication deterministe : `row_number()` `over(partition by app_id order by updated_ts_utc desc nulls last)` puis `where row_num = 1`. Cette correction a ete introduite apres un incident detecte pendant les essais SCD2, ce qui renforce la robustesse de la couche la plus fragile.

3.4 Figure Lab 2 : top applications

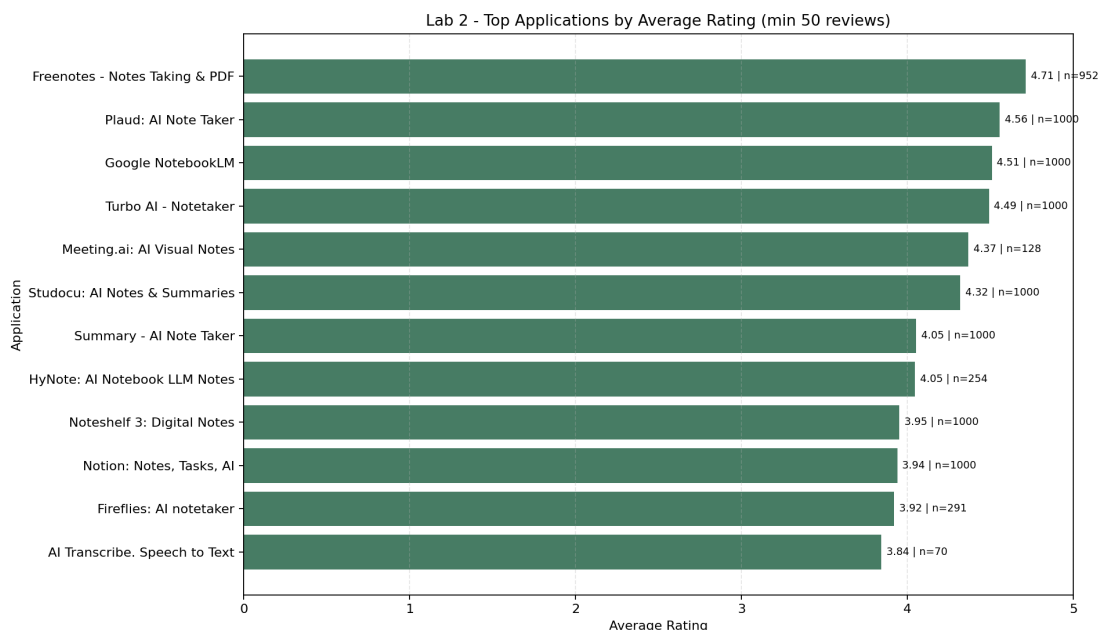


FIGURE 5 – Lab 2 - Top applications par note moyenne (seuil minimum de reviews).

Lecture détaillée : cette figure montre les applications en tête quand on impose un minimum de volume pour éviter les faux champions. Exemple : *Freenotes* est en haut avec 4.714 sur 952 reviews, *Plaud* suit avec 4.556 sur 1000. Le choix d'un seuil est un point de rigueur analytique : sans ce filtre, des apps avec 5 ou 10 reviews pourraient apparaître artificiellement excellentes.

3.5 Figure Lab 2 : tendance mensuelle

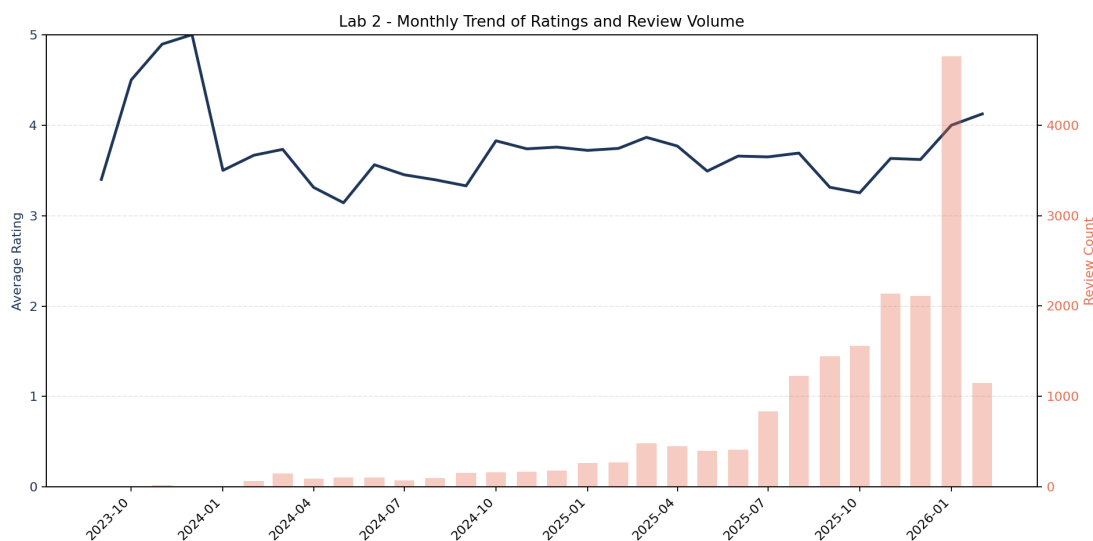


FIGURE 6 – Lab 2 - Tendance mensuelle de la note moyenne et du volume de reviews.

Lecture detaillee : la tendance mensuelle montre une relation non lineaire entre volume et satisfaction. Exemples chiffrés : en 2025-10, le volume monte à 1563 reviews mais la note moyenne reste basse (3.252) ; en 2026-01, le volume est très élevé (4761) avec une note qui remonte (3.999). Cela justifie l'utilisation conjointe des deux indicateurs dans l'analyse.

4 Implementation detaillee des points techniques critiques

4.1 Incremental loading (implementation + preuve)

Implementation dans LAB 2/lab2_playstore/models/marts/fct_playstore_reviews.sql.

```
1 {{ config(materialized='incremental', unique_key='review_id') }}
2 ...
3 {% if is_incremental() %}
4 where review_ts > (
5   select coalesce(max(review_ts), cast('1900-01-01' as timestamp))
6   from {{ this }}
7 )
8 {% endif %}
```

Listing 1 – Extrait simplifie de la logique incremental dans la fact.

Preuve réelle issue de nos artefacts :

- avant simulation : 18898 lignes dans la fact,
- apres injection d'une review de test : 18899,
- delta constate : +1 (pas de rebuild complet).

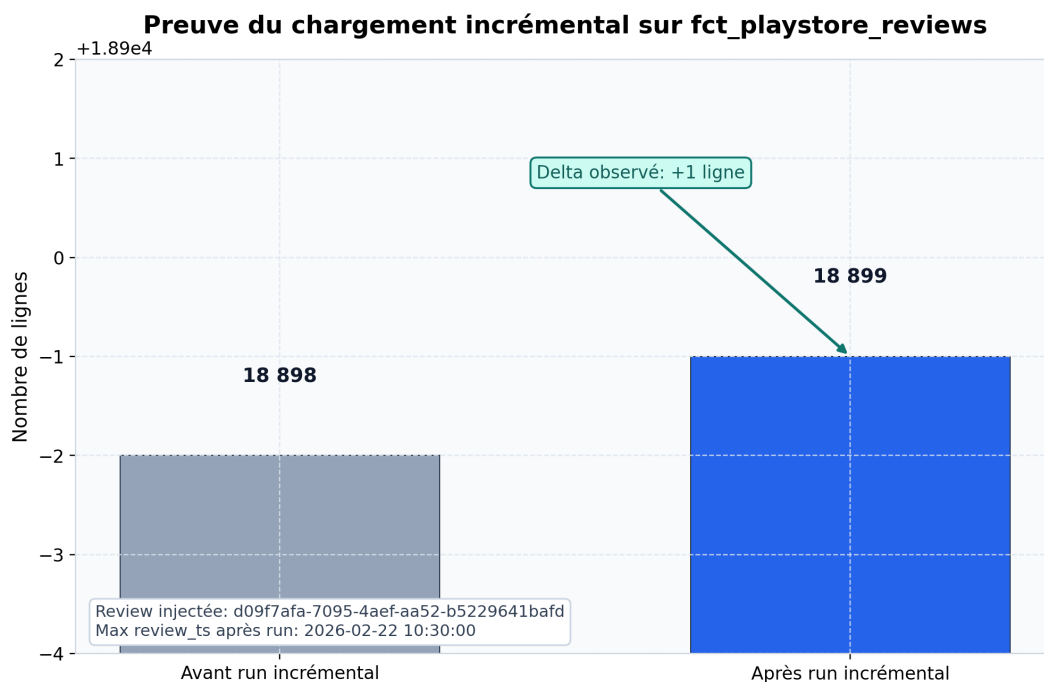


FIGURE 7 – Preuve incremental : la fact evolue de 18898 a 18899 (+1).

Lecture detaillee : la figure ne montre pas un simple comptage, elle valide le comportement attendu d'un modele incremental : isoler un delta minimal sans recharger tout l'historique.

4.2 SCD2 snapshot (implementation + preuve)

Implementation dans LAB 2/lab2_playstore/snapshots/snap_dim_apps_scd2.sql avec `strategy='check'` et `unique_key='app_id'`.

Le principe applique : lors d'un changement d'attributs sur une app, dbt ferme l'ancienne ligne (`dbt_valid_to`) et cree une nouvelle version active.

Preuve reelle :

- app cible : `com.google.android.apps.labs.language.tailwind`,
- lignes snapshot : 30 -> 31,
- versions pour cette app : 2.

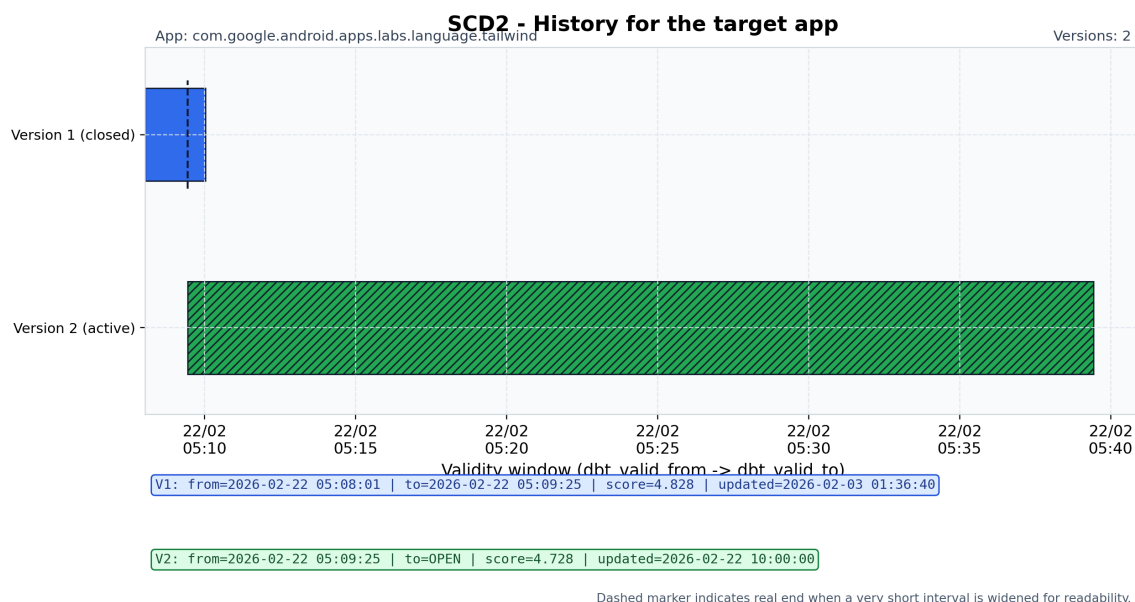


FIGURE 8 – Preuve SCD2 : historique temporel sur l'app cible (version fermee + version active).

Lecture detaillee : la figure montre exactement ce qu'on attend d'un SCD2 : une version historique cloturee et une version courante ouverte. C'est la condition necessaire pour faire des analyses *as-of* sans perdre l'etat actuel.

4.3 Data quality & testing (implementation + preuve)

Les tests sont declares dans :

- `models/staging/schema.yml`,
- `models/marts/schema.yml`,
- `snapshots/schema.yml`,
- et tests SQL custom dans `tests/`.

Sur notre execution documentee dans LAB1_Python-DATAPipeline-/LAB 2/lab2_playstore/target nous avons 49 tests, 49 pass, 0 warn, 0 fail.

Résultat global des tests dbt

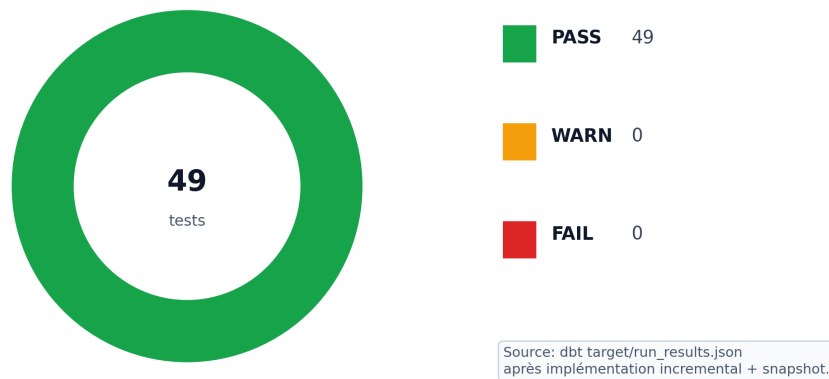


FIGURE 9 – Synthese des tests dbt sur l'execution complete.

Lecture detaillee : ce resultat n'est pas une formalite. Il valide les contraintes structurantes (`not_null`, `unique`), l'integrite relationnelle (`relationships`) et les regles de domaine (valeurs autorisees, bornes). Sans ce filet, les regressions en staging ne seraient detectees qu'en aval.

4.4 Volumes modeles et coherence de grain

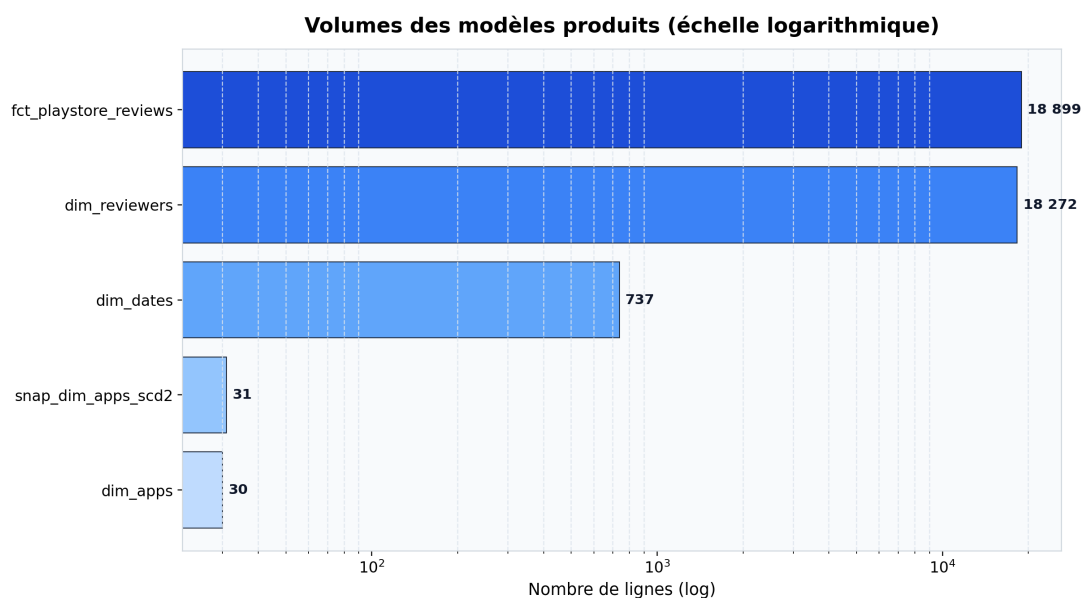


FIGURE 10 – Volumes des modeles principaux produits dans la couche analytique.

Lecture detaillee : les volumes observes sont coherents avec la structure du modele : `dim_apps=30`, `dim_dates=737`, `dim_reviewers=18272`, `fct_playstore_reviews=18899`,

snap_dim_apps_scd2=31. Le ratio reviewers/fact et la croissance moderee du snapshot confirment une granularite maîtrisee.

5 Comparaison pipeline Python-only vs pipeline dbt

Aspect	Lab 1 (Python-only)	Lab 2 (dbt + DuckDB)
Logique de transformation	Scripts pandas flexibles	Modeles SQL versionnes et documentables
Vitesse de depart	Tres rapide	Un peu plus structurant au debut
Controle qualite	Plutot manuel	Tests declaratifs centralises
Historisation	Non native	Snapshot SCD2 natif
Incremental loading	Ad-hoc possible	Natif, lisible, auditable
Maintenance	Risque de dette en croissance	Plus stable a moyen/long terme
Traçabilité	Forte discipline necessaire	Native via artefacts dbt

TABLE 3 – Comparaison appliquee a notre cas reel Play Store.

Conclusion de la comparaison : le Lab 1 etait la bonne strategie pour lancer vite le sujet et valider les flux. Le Lab 2 etait necessaire pour passer a un niveau production-analytique (qualite verifiable, historisation, tests reproductibles).

6 Reflexions critiques sur nos choix d'architecture

6.1 Partie la plus fragile de la pipeline

La partie la plus fragile a ete la frontiere *raw* -> *staging*. Les incidents de schema drift, de valeurs invalides et de doublons arrivent d'abord a cet endroit. Notre cas concret le plus parlant : pendant les essais SCD2, un doublon d'app a temporairement casse une contrainte d'unicite en staging, ce qui a impose la regle de deduplication deterministe actuelle.

6.2 Plus grand insight d'architecture

Le plus grand apprentissage est que la fiabilite ne vient pas d'un code "qui tourne", mais d'un systeme qui explicite ses hypotheses et detecte ses propres ecarts. Le couple *modeles + tests + artefacts* a transforme notre pipeline d'un prototype fonctionnel vers un actif analytique defendable.

6.3 Une decision de design que nous changerions

Si nous repartions de zero, nous stabiliserions plus tot la gestion des chemins d'entree et la registration des sources brutes pour reduire la dependance au contexte d'execution local. Ce point devient critique des que plusieurs environnements (poste local, CI, serveur) doivent partager la meme pipeline.

7 Tracabilite : modeles, sorties et preuves utilisees

7.1 Modeles et scripts references

- `src/ingest.py`, `src/transform.py`, `src/serve.py`, `src/stress_test.py`
- `LAB 2/lab2_playstore/models/staging/stg_playstore_apps.sql`
- `LAB 2/lab2_playstore/models/marts/fct_playstore_reviews.sql`
- `LAB 2/lab2_playstore/snapshots/snap_dim_apps_scd2.sql`

7.2 Fichiers de preuve utilises dans ce rapport

- `data/processed/stress_test_report.md`
- `data/processed/quality_report.json`
- `data/processed/lab_report_baseline_metrics.json`
- `data/processed/lab_report_post_metrics.json`
- `data/processed/lab_report_delta_summary.json`
- `data/processed/lab_report_model_row_counts.csv`
- `data/processed/lab_report_scd2_target_app.csv`
- `LAB 2/lab2_playstore/target/run_results.json`

8 Conclusion generale

Nous avons repondu a l'ensemble des objectifs des deux labs avec une progression coherente : construction, stress testing, puis industrialisation. La version finale livre :

- un pipeline Lab 1 robuste et instrumente,
- un pipeline Lab 2 modelise en schema etoile,
- un incremental loading prouve,
- un SCD2 prouve,
- une qualite des donnees testee de maniere automatisable,
- une analyse critique argumentee de nos choix techniques.

L'enjeu n'etait pas seulement d'obtenir un resultat, mais de produire une architecture que nous pouvons justifier techniquement avec des preuves concretes.

References

- README principal : LAB1_Python-DATAPipeline-/README.md
- Rapport de reflection detaille : LAB1_Python-DATAPipeline-/LABS_REFLECTION_REPORT.md
- Journal de progression Lab 1 : LAB1_Python-DATAPipeline-/LAB_PROGRESS_LOG.md
- Journal Lab 2 : LAB 2/LAB2_PROGRESS_LOG.md
- Plan de mise en oeuvre Lab 2 : LAB 2/LAB2_STEP_BY_STEP_PLAN.md